Final Project Design Phase

Computer Science - Week 11

Oct 3, 2026 - Version 0.0.4

For complete instructions and project ideas, refer to the assignment in Canvas. Specifically, review the final project requirements before you continue below. If you have not reviewed the project requirements in Canvas (you might have to go to the modules view and check the current or past week's content), you will not be able to properly complete the parts below. In addition, it may be helpful to review the information on this Drafter documentation page.

0) Project Planning and Preparation

Successful programmers spend time planning before they begin coding. *After* reviewing the project requirements, answer the following questions to help guide your design and development process.

A. What concepts from class or previous assignments might help you complete this project? Which programming concepts (loops, conditionals, lists, classes, etc.) do you expect to use? What programming skills do you want to improve while completing this project?

Concepts that will help me complete the project are user input, math operations such as addition, subtraction and multiplication, writing conditional statements, and dataclasses. The programming skill I'd want to improve while completing this project is working with nested data from dataclasses. I was close to passing exam codes of those types of problems, but I could not get all of them.

B. Which parts of the project do you expect to be the easiest for you? The hardest? Why?

The easiest part of the project would be writing the user inputs and dataclasses, and conditionals. The key is to not overcomplicate them. The most difficult part of the project will be making sure drafter interface is functioning properly. I'd personally say this for me because I'm still a little uncomfortable working with drafter. The back-end code should be easier.

C. Even though you have not yet designed the final project, estimate which tasks will likely require the most time and effort? Which features or components should be implemented first?

The task that should take the most time and effort is going to be crosschecking to make sure my idea matches up with what the LLM produces. I plan to chunk code my website first, then check that to make sure I'm meeting the requirements for the project. I plan to write out the dataclasses, user input statements, and algebraic operations in this chunk format. That entire process will take the longest time. The LLM will then convert this to code. Data classes and user input statements should be implemented first.

D. What distractions might interfere with your work and how will you minimize them? What resources will you need available while working? What questions, if any, do you still have about the assignment requirements?

Distractions that could interfere with my work is my heavy workload from my other classes. I have a loaded week next week with 3 exams, so I am more than likely going to prioritize studying for those over working on this project, at least for an elongated period. I can minimize work interference with this project by working on it in small chunks at a time knowing that most of my time will be spent on my other classes. Small work sessions will eventually add up and accumulate to a finished, polished product.

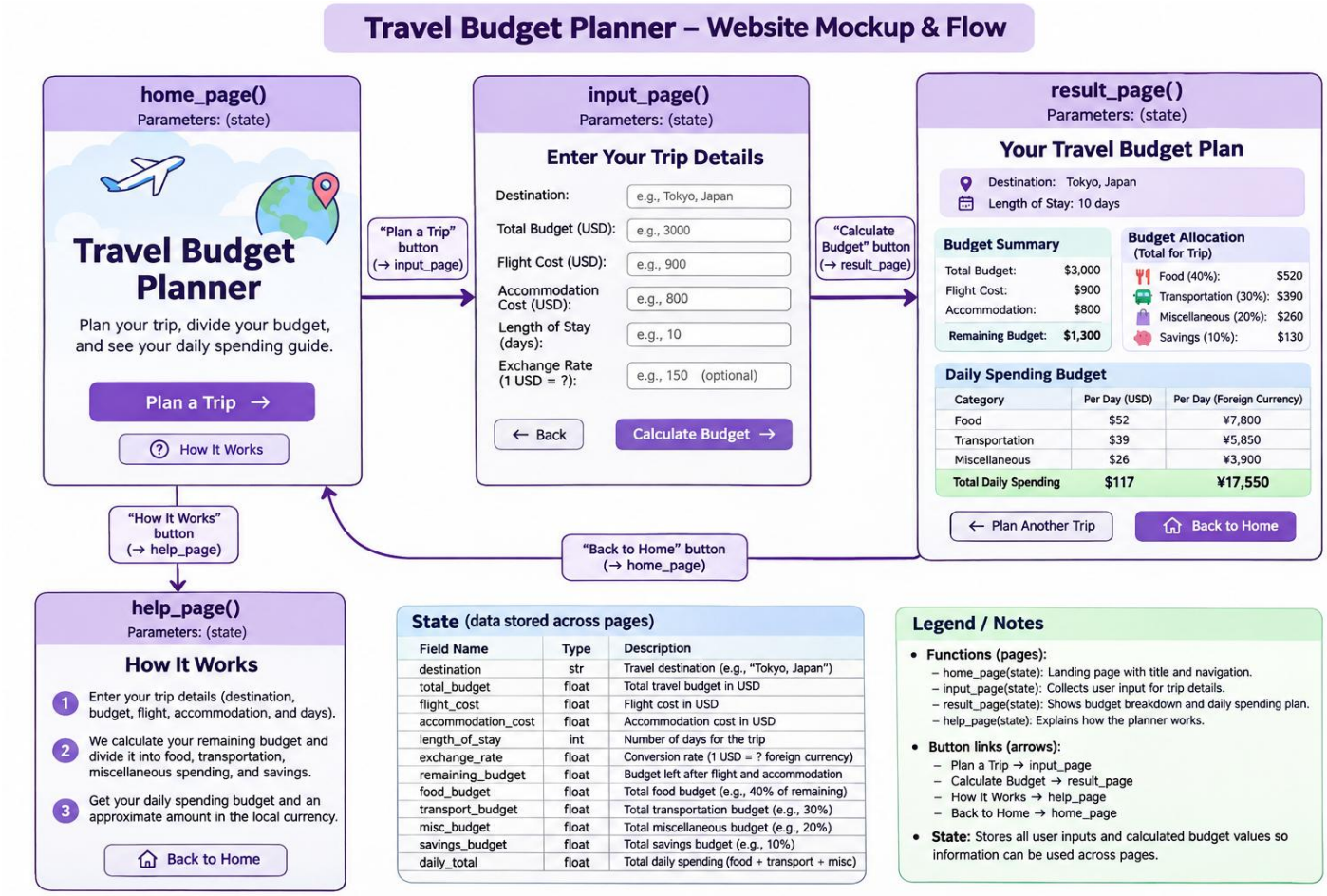
1) Site Diagram

Create a mock-up of your planned website. We suggest you make the mockup on paper, although you can also use software like PowerPoint or Slides to make a mockup. There is other design software that could also be used, but we strongly encourage you to keep it simple. Do not spend more than 15-30 minutes detailing the mock-up. Your goal should be to make a very quick and dirty diagram, to show that you have some kind of idea.

We recommend drawing each page as a rectangle, and laying out the text, buttons, and other components inside. Then, use arrows to indicate links between pages. Label each page the way you will name each function, and then put labels on the arrows to indicate what the button links. Also write down any parameters that should come into the pages.

In addition to the pages and routes, write down what fields your `State` will have, both their names and types. Think clearly about what data will need to be persistent across the entire website.

Add photo(s) or screenshot(s) of your website mock-up and labeled diagram on the next page.



Field Functions:

Page	Function Name	Parameter
Home Page	home_page	state: State
Trip Input Page	input_page	state: State
Budget Results Page	result_page	state: State
How It Works Page	help_page	state: State

Button Navigation:

Button	Starting Page	Destination
Plan a Trip	Home	Input
Calculate Budget	Input	Results
Back	Input	Home
How It Works	Home	Help
Back to Home	Help	Home
Plan Another Trip	Results	Input
Back to Home	Results	Home

State Fields and Types:

Field Name	Python Type	Purpose
destination	str	Travel destination
total_budget	float	Total available money
flight_cost	float	Round-trip flight expense
accommodation_cost	float	Total lodging expense
length_of_stay	int	Number of travel days
exchange_rate	float	Foreign currency per USD
remaining_budget	float	Money after fixed expenses
food_budget	float	Money allocated to food
transport_budget	float	Money allocated to transportation
misc_budget	float	Money allocated to miscellaneous expenses
savings_budget	float	Money reserved for emergencies
daily_total	float	Daily spending allowance

2) Implementing Requirements

This table should serve as a draft for how you will implement the requirements into your final project. You may change your ideas later on, but your final site must fulfill all the requirements.

Requirement	**Implementation**

Purpose/Functionality A 1-2 sentence description of the purpose or functionality of your site	**The purpose of my site is to help individuals financially plan for daily spending during a trip/vacation. They will be able to allocate money to specific categories, so they are aware of their spending.**
State Dataclass Attributes Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass. Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.	Destination: str that stores travel destination total_budget: float that stores travel budget in USD flight_cost: float that stores cost of flight accommodation_cost: float that stores cost of living accommodation expenses: list[Expense]: list of dataclass that stores a list of pre-planned expenses entered by user.
Second Dataclass Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)	Second Dataclass is Expense Its attributes are name: str (name of expense), category: str (food, transportation, or miscellaneous spending), and cost: float (cost of expense) When a planned expense is entered, an expense object is created and added to the expenses list in State.
Routes (minimum 7) The names of your route functions	Home, Calculator, Initial Report, Add Expenses, Save Expenses, View Report, Back to Results, Back to Home
Pages (minimum 5) A brief description of the pages in	The 5 pages are the home page, trip input page, budget results page, expense report page, and add expense page. The welcome page is just and opener and breaks down how the website functions, the trip

your site	input pages is where the user inputs all important information such as flight and accommodation cost, budget result page displays money and suggested partitioning, the expense report page gives final breakdown of trip expenses, and finally the add expenses page allows users to add money and edit how they want to allocate their remaining funds.
User Input (minimum 3) Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes For more information on these input types, see Drafter Documentation.	The main user input the program cares about is the users total budget, flight cost, accommodation cost, exchange rate, and length of stay. Those are all floats/ ints. The only string user input is the destination.
If Statements (minimum 3) Brief description of what will be evaluated in your if statements and how they will meaningfully change the logic of the site	3 key if statements include checking the budget, checking the trip length, and checking available daily spending. The available daily allowance sends a warning when daily budget is below $50. The check trip length is a guard to make sure the trip is a positive amount of days. The check budget displays a warning if the budget is less than the flight cost and accommodation cost.
Loop (minimum 1) This loop must iterate over the list/dict of dataclass instances in your State attribute. Description of this loop and why it is meaningful to the site	One for loop that adds the cost of each expense to calculate the total. This allows the user to see how much money they are spending when it's all said and done. This is they key component of the program.

LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top

of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

Me: can you convert a chunk code table into python code if i upload it

LLM: Yes! Absolutely. ☑

*If you upload your **chunk code table**, I can convert it into working **Python code** for your Travel Budget Planner project.*

I can also:

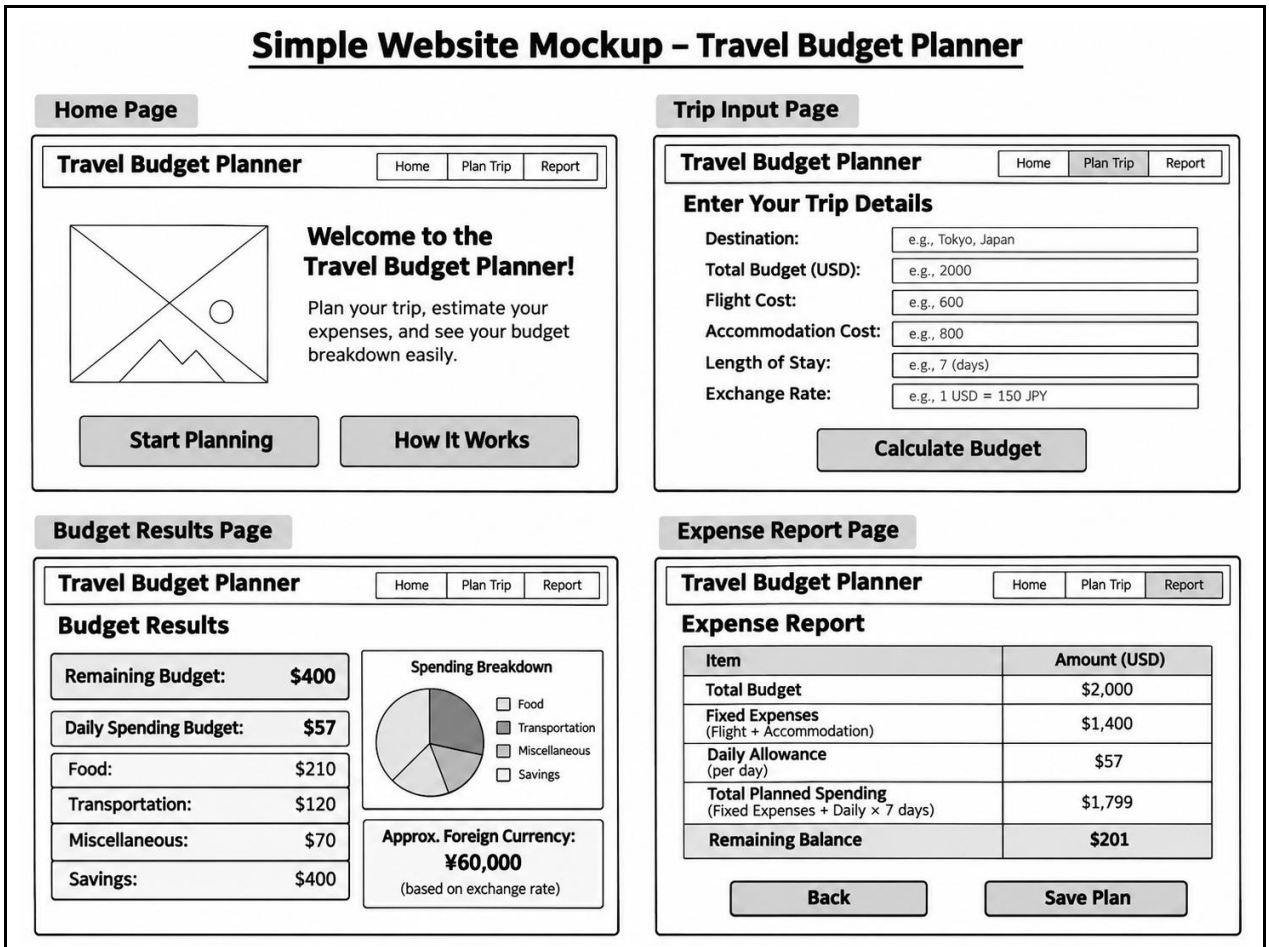
- ☑ *Turn each chunk into a Python function.*
- ☑ *Combine the chunks into one working program.*
- ☑ *Use beginner-friendly Python appropriate for CS1.*
- ☑ *Adapt the code to work with **Drafter** for your web application.*
- ☑ *Add comments explaining what each function does.*
- ☑ *Help debug errors and fix problems.*

***Upload your chunk code table** as a PDF, Word document, screenshot, or image.*

I'll follow your table as closely as possible and explain how each chunk translates into Python so you understand the code, too.

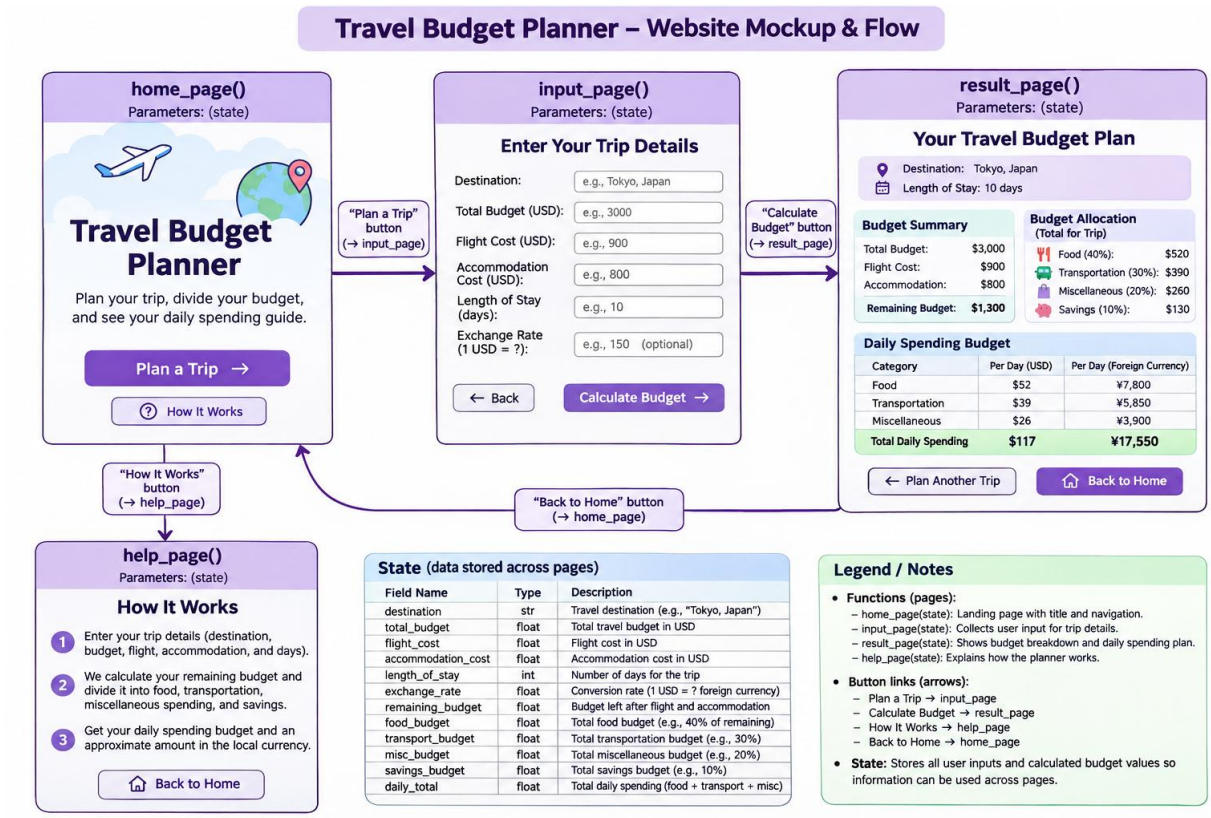
Me: now i need to maka a mock up of the planned website simple, with each page as a rectangle

LLM:



Me: lay out the text, buttons, and other components inside. Then, use arrows to indicate links between pages. Label each page the way you will name each function, and then put labels on the arrows to indicate what the button links. Also write down any parameters that should come into the pages. write down what fields your `State` will have, both their names and types

LLM:



Me: need 3 if statements, 1 loop, min 3 user inputs, min 5 pages, min 7 routes, state data class with 4 attributes with one must be a list/dict of instances of second data class. need a second dataclass. including its attributes, types, and how they will meaningfully be modified,1-2 sentence purpose/functionality briefer of site

LLM: My Travel Budget Planner is a website that helps users organize their travel expenses by calculating how much money they can spend each day based on their total budget, flight cost, accommodation cost, and length of stay. The website will divide the remaining money into food, transportation, miscellaneous spending, and emergency savings while providing an expense report and approximate foreign currency conversion.

Page	Function Name	Purpose
1. Home Page	`home_page`	Welcomes users and explains the website.
2. Trip Input Page	`input_page`	Collects destination, budget, flight cost, accommodation cost, trip length, and exchange rate.
3. Budget Results Page	`results_page`	Displays remaining money and suggested category budgets.

4. Expense Report Page	`report_page`	Shows daily spending limits and a breakdown of trip expenses.
5. Add Expenses Page	`expenses_page`	Allows users to add and manage planned travel expenses.

Route	Starting Page	Button	Destination
1	Home	Start Planning	Input Page
2	Input	Calculate Budget	Results Page
3	Results	View Report	Expense Report
4	Results	Add Expenses	Add Expenses Page
5	Expenses	Save Expense	Expenses Page (updated)
6	Expenses	View Report	Expense Report
7	Report	Back to Results	Results Page
8	Results	Back to Home	Home Page

Three If Statements

Your project can use conditional statements to check whether the user's budget is reasonable.

If Statement 1: Check Budget

Purpose: Determine whether the traveler can afford the flight and accommodation.

```
if total_budget < flight_cost + accommodation_cost:    # Display an over-budget warning
```

If Statement 2: Check Trip Length

Purpose: Prevent errors when calculating daily spending.

```
if length_of_stay <= 0:    # Display an error asking for a valid number of days
```

If Statement 3: Check Available Daily Spending

Purpose: Warn users when their daily spending allowance is low.

For example, your program can use $50 as a sample threshold.

```
if daily_budget < 50:    # Display a warning about limited daily spending
```

You could also use an else branch to display a positive message when the budget meets that threshold.

☑ **3 meaningful conditional checks.**

---

5. One Loop

Your loop will process the list of planned expenses.

For example, a user adds:

Expense	Cost
Museum	$25
Train Pass	$40
Tour	$60

Your loop goes through the list and adds each expense's cost to calculate the total.

total_expenses = 0for expense in state.expenses: total_expenses += expense.cost

Purpose: Calculate the total cost of all planned activities and additional expenses.

You can then subtract the total planned expenses from the available spending budget to show how much money is still uncommitted.

☑ **1 meaningful loop that works with your second data class.**

---

6. User Inputs

Your application will have at least six inputs.

Input	Data Type	Example
Destination	str	Tokyo, Japan
Total Budget	float	3000
Flight Cost	float	900
Accommodation Cost	float	800
Length of Stay	int	10
Exchange Rate	float	150

The Expenses Page adds two more inputs: expense name and expense cost.

☑ **8 user inputs — exceeds the minimum of 3.**

---

7. First Data Class: State

Your main data class will be called State.

This stores the information entered by the user and allows it to be used across different pages.

Attribute	Type	How It Changes
destination	str	Updated when users enter or change their travel destination.
total_budget	float	Updated when users enter their available budget.
flight_cost	float	Updated when users enter their flight expense.
accommodation_cost	float	Updated when users enter their accommodation expense.
length_of_stay	int	Updated when users enter the number of days they will travel.
exchange_rate	float	Updated when users enter an approximate currency exchange rate.
expenses	list[Expense]	Updated whenever users add a planned expense.

Example State Data Class

@dataclassclass State: destination: str total_budget: float flight_cost: float accommodation_cost: float length_of_stay: int exchange_rate: float expenses: list[Expense]

You have **7 attributes**, including a list of instances of your second data class.

☑ Meets the requirement of at least 4 attributes.

---

8. Second Data Class: Expense

Your second data class will represent an individual planned expense.

For example, if a user wants to visit a museum, the program can create an Expense object containing the name, category, and cost of that activity.

Expense Data Class

@dataclassclass Expense: name: str category: str cost: float

Attributes and Types

Attribute	Type	Purpose
name	str	Stores the name of the planned expense.
category	str	Stores whether the expense is food, transportation, or miscellaneous.
cost	float	Stores the expense amount in USD.

How Will the Attributes Be Modified?

Name: When users enter a new planned expense, the program stores its name, such as "Museum Ticket."

Category: Users select which spending category the expense belongs to.

Cost: Users enter the expense cost in USD. The program uses this amount to calculate how much of the budget is committed.

When a user adds another expense, the program creates a new Expense object and adds it to the expenses list stored in State.

For example:

new_expense = Expense("Museum Ticket", "Miscellaneous", 25.0)state.expenses.append(new_expense)

This is a meaningful use of data abstraction because each expense has its own information.

☑ **Two data classes with a meaningful relationship.**

---

9. How the Budget Calculations Will Work

Imagine someone enters:

Information	Amount
Total Budget	$3,000
Flight	$900
Accommodation	$800
Length of Stay	10 days

The program first calculates:

Remaining Budget = $3,000 − $900 − $800 = $1,300

It then uses the 40/30/20/10 budgeting rule.

Category	Percentage	Budget
Food	40%	$520
Transportation	30%	$390
Miscellaneous	20%	$260
Emergency Savings	10%	$130

The daily spending budget would be:

($520 + $390 + $260) ÷ 10 = **$117 per day**

Now imagine the traveler adds three planned expenses:

Expense	Category	Cost
Museum Ticket	Miscellaneous	$25
Train Pass	Transportation	$40
Restaurant Reservation	Food	$60
Total		**$125**

The program loops through the expenses and calculates a total of $125.

It can then display:

Daily Spending Allowance: $117

Total Planned Additional Expenses: $125

Remaining Uncommitted Spending Budget: $1,045

The $125 in planned expenses is part of the original $1,170 spending allowance, so it should not be subtracted from the full $1,300 budget twice.

This also gives you something useful to display on your Expense Report Page.

---

10. Final Requirements Checklist

Requirement	Your Project	Met ?
At least 3 if statements	Budget check, trip length check, daily spending warning	☑
At least 1 loop	Loop through planned expenses	☑
At least 3 user inputs	8 user inputs	☑
At least 5 pages	Home, Input, Results, Report, Expenses	☑
At least 7 routes	8 navigation/action routes	☑
State data class	7 attributes	☑
List of second data class objects	expenses: list[Expense]	☑
Second data class	Expense	☑
Meaningful data modifications	User updates trip information and adds expenses	☑
Purpose/functionality description	2 sentences	☑

My Recommendation

For your CS1 final project, **keep the five pages and eight routes described above**.

The most important addition is the **Add Expenses Page** because it naturally connects several of your requirements:

- Your second data class (Expense) represents individual expenses.
- Your first data class (State) stores a list of those expenses.
- Your loop calculates the total cost of the expenses.
- Your expense report displays the information.

This creates a connected application instead of adding unrelated features just to meet the assignment requirements.

Me: is it ok to name my routes with the button names

LLM: Yes! ☑ **You can name your routes after the buttons**, and that's actually a good idea for your CS1 Drafter project.

Just make sure you use **Python naming conventions**:

- Use lowercase letters.
- Replace spaces with underscores (_).
- Make each route name unique.
- Choose names that describe what the button does.

Example: Your Travel Budget Planner

Button Name	**Route Function Name**	**Destination**
Start Planning	start_planning	Input Page
Calculate Budget	calculate_budget	Results Page
View Report	view_report	Report Page
Add Expenses	add_expenses	Expenses Page
Save Expense	save_expense	Expenses Page (updated)
View Report	view_expense_report	Report Page
Back to Results	back_to_results	Results Page
Back to Home	back_to_home	Home Page

Important Difference: Pages vs. Routes

In Drafter, **page functions display information**, while **route functions handle actions and navigation**.

For example:

@routedef start_planning(state: State) -> Page: return input_page(state)

Here:

- start_planning is the **route function**.
- input_page is the **page function**.
- state stores your application's information.

Your button would use:

Button("Start Planning", start_planning)

My Recommendation

For your website mockup, label your arrows with the **button name and corresponding route function**.

For example:

Home Page → Input Page

Arrow label: Start Planning (start_planning)

This makes it easy for your professor to see how the buttons connect to the routes.

☑ **One thing to remember:** If multiple buttons have the same name, their route functions can still have different names based on their purpose.

You can absolutely use this approach for your final project!

Me: data class attributes?

LLM: For your **Travel Budget Planner**, you need **2 data classes**. The first is State, which stores the user's travel information, and the second is Expense, which stores individual planned expenses.

1. First Data Class: State

Attribute Name	Type	Purpose
destination	str	Stores the user's travel destination.
total_budget	float	Stores the user's total travel budget in USD.
flight_cost	float	Stores the cost of the flight.
accommodation_cost	float	Stores the total accommodation cost.
length_of_stay	int	Stores the number of days traveling.
exchange_rate	float	Stores the exchange rate between USD and the destination's currency.
expenses	list[Expense]	Stores a list of planned expenses entered by the user.

Python Representation

from dataclasses import dataclass, field@dataclassclass State: destination: str = ""

total_budget: float = 0.0 flight_cost: float = 0.0 accommodation_cost: float = 0.0 length_of_stay: int = 1 exchange_rate: float = 1.0 expenses: list["Expense"] = field(default_factory=list)

How the attributes change: Users enter or update their trip information on the Input Page. The expenses list changes whenever users add a planned expense.

---

2. Second Data Class: Expense

This data class represents individual planned travel expenses.

Attribute Name	Type	Purpose
name	str	Stores the name of the expense.
category	str	Identifies whether the expense is food, transportation, or miscellaneous.
cost	float	Stores the cost of the expense in USD.

Python Representation

@dataclassclass Expense: name: str category: str cost: float

How the attributes change: When a user enters a planned expense, the program creates an Expense object containing its name, category, and cost. That object is added to the expenses list in State.

---

3. How the Two Data Classes Connect

For example, imagine someone is planning a trip to Japan and adds these expenses:

Name	Category	Cost
Sushi Dinner	Food	$35
Subway Pass	Transportation	$20
Museum Ticket	Miscellaneous	$25

Each expense becomes an Expense object.

The State data class stores these objects in its expenses list.

Your program can then use a **for loop** to calculate their combined cost.

☑ **How This Meets Your Assignment Requirements**

Requirement	Your Project
First data class	State
Minimum 4 attributes	7 attributes
At least one list or dictionary	expenses: list[Expense]
Second data class	Expense
Second data class attributes	name, category, cost
Meaningful modifications	Users update trip details and add planned expenses.

My recommendation: Keep these two data classes. They are straightforward, meet your CS1 requirements, and fit naturally into your Travel Budget Planner.

Final Submission

When you are finished, download this file as a Word Document (docx) and upload the file to the appropriate assignment on Canvas.